
System Overview

Team : AISC 5 (VidChap)

Name : Cliphop

The system allows a user to paste a YouTube link and generates segmented chapters with titles and timestamps by:

1. Extracting audio from the YouTube video.
2. Transcribing the audio using Whisper API.
3. Using Groq LLM API to segment and title chapters with timestamps.
4. Displaying the chapters on a frontend along with a YouTube video player.
5. Deploying the entire system on Render using GitHub hooks for CI/CD.

System Components

1. Frontend

Responsibilities:

Allow users to input a YouTube link.

Display segmented chapters (titles & timestamps).

Embed a YouTube video player for playback.

Allow users to click chapter names to skip to timestamps.

Tech Stack:

React: For building the UI.

YouTube Player API: To display and control the video player.

Key Features:

Input field for the YouTube link.

Dynamic list rendering of chapters (titles & timestamps).

Integration with the YouTube player for skipping to timestamps.

2. Backend

Responsibilities:

Extract audio from the YouTube video.

Transcribe audio using Whisper API.

Process transcript using Groq LLM API for chapter segmentation.

Provide endpoints for the frontend to fetch chapter data.

Tech Stack:

Node.js (Express): For building APIs.

FFmpeg: For extracting audio from YouTube videos.

Python Scripts: For Whisper transcription and Groq LLM API integration.

API Endpoints:

1. POST /process-video:

Input: YouTube link.

Output: JSON response with chapter data (titles & timestamps).

2. GET /chapters/:videoid:

Input: Video ID.

Output: Chapter data for the requested video.

3. External APIs & Libraries

1. YouTube API:

Fetch video metadata.

Embed the video player in the frontend.

2. FFmpeg:

Extract audio from YouTube video.

3. Whisper API:

Transcribe audio into text.

4. Groq LLM API:

Generate segmented sections with titles and timestamps from the transcript.

5. Database

Responsibilities:

Store chapter data for previously processed videos.

Avoid reprocessing the same video link.

Tech Stack:

PostgreSQL (preferred for structured data).

Schema Design:

Videos Table:

Id: Primary Key.

Video_id: YouTube video ID.

Title: Video title.

Chapters: JSON field containing titles and timestamps.

Processed_at: Timestamp of processing.

6. CI/CD Pipeline

Responsibilities:

Automate build, test, and deployment processes.

Deploy changes to Render upon GitHub commits.

Tools:

GitHub Actions: For CI/CD.

Render: For deployment of the frontend, backend, and database.

Workflow:

1. Push code to GitHub.

2. GitHub Actions triggers:

Build: Install dependencies, lint, and test.

Deploy: Deploy to Render (frontend, backend, and database services).

7. Deployment

Platform: Render

Frontend Service: React app deployed as a static site.

Backend Service: Node.js app with Python scripts, deployed as a web service.

Database: MongoDB

Flow of Execution

1. User Input:

User pastes a YouTube link in the frontend.

2. Audio Extraction:

Backend uses FFmpeg to extract audio from the YouTube video.

3. Transcription:

Audio is sent to the Whisper API for transcription.

4. Chapter Segmentation:

Transcript is processed by the Groq LLM API to generate chapters with titles and timestamps.

5. Data Storage:

Chapter data is saved in the database for future use.

6. Frontend Display:

The frontend fetches the chapter data and displays it below the YouTube player.

7. User Interaction:

Users can click chapters to skip to the corresponding timestamp using the YouTube API.

8. Deployment:

CI/CD pipeline automatically deploys the system to Render.

System Architecture Diagram

Modules:

Frontend (React)

Backend (Node.js + Python)

Database (MongoDB)

External APIs (YouTube API, Whisper API, Groq LLM API)

1. Frontend: Input form → Fetch API → Render video player & chapters.

2. Backend:

Extract audio → Transcribe → Generate chapters → Serve chapter data.

3. Database: Store processed video data.

4. External APIs:

Whisper API: Convert audio to transcript.

Groq LLM API: Generate chapters from transcript.

YouTube API: Embed player & fetch video data.

5. Render: Deploy services using GitHub Actions.
